

PDF 4: Arquitectura de Persistencia, Modelo ACID y Transaccionalidad

Este documento define la capa de persistencia, las relaciones de las entidades y el protocolo de integridad de transacciones para el servidor MySQL.

1. Esquema Relacional y Entidades Críticas

El modelo sigue una estrategia de mapeo Objeto-Relacional (ORM) mediante **Hibernate/JPA**, garantizando la integridad referencial y la normalización de datos.

A. Tabla **usuarios** (La Fuente de Verdad)

- **Identificadores:** **id** (PK, BigInt, AutoIncrement), **nombre_usuario** (Unique, Not Null), **correo_usuario** (Unique).
- **Seguridad:** **contrasenha_usuario** almacena el hash Bcrypt (no el texto plano).
- **Estado Agregado:** Contiene campos contadores (**frases_acertadas**, **partidas_jugadas**, **mejor_puntuacion_contrarreloj**) que actúan como caché de resultados. Esto evita consultas **COUNT** o **SUM** pesadas sobre la tabla de **partidas** cada vez que se carga el dashboard.
- **Personalización:** **avatar_config** almacena el *seed* de la API DiceBear, permitiendo reconstruir la identidad visual del usuario sin almacenar imágenes binarias (BLOBs).

B. Tabla **partidas** (Telemetría y Registro Histórico)

- **Relación 1:N:** Definida con **@ManyToOne** hacia la entidad **usuarios** y **@OneToMany** desde **usuarios** hacia **partidas**.
- **Telemetría:** Registra cada sesión: **puntuacion**, **creditos_ganados**, **palabras_correctas**, **precision** (float) y **modo_juego** (Enum: 'ROSCO', 'CONTRARRELOJ', 'VERSUS').
- **Optimización:** El uso de índices en la columna **usuario_id** es crítico para recuperar el historial de un piloto específico sin realizar escaneos completos de tabla (*Full Table Scans*).

C. Tabla **incidencias** (Observabilidad y Auditoría)

- **Propósito:** Actúa como el *Log centralizado* de errores.
- **Atributos:** **timestamp**, **endpoint_origen**, **clase_afectada**, **mensaje_error** y **stack_trace**.

- **Independencia:** Esta tabla no tiene llaves foráneas (FK) hacia la lógica de negocio, lo que permite que el sistema de auditoría funcione aunque la lógica transaccional de un usuario falle.

2. Gestión Económica: Transacciones ACID

La economía del juego se basa en una estricta política de **Transaccionalidad ACID** (Atomicity, Consistency, Isolation, Durability) gestionada mediante el contenedor de Spring.

El Ciclo de Vida de una Transacción (@Transactional)

Cualquier operación que modifique la billetera del usuario o su inventario está protegida por la etiqueta @Transactional.

- **Atomicity (Atomicidad):** "Todo o nada". Si el proceso de compra de un objeto implica:
 - Validar saldo (AppState).
 - Restar créditos de la fila usuarios.
 - Insertar una fila en inventario (o tabla puente usuario_item).
 - Si el sistema sufre un corte eléctrico o un error de base de datos en el paso 2.5, **no se ejecuta el paso 1 ni el 3**. La base de datos realiza un **Rollback** automático, devolviendo al usuario sus créditos.
- **Consistency (Consistencia):** Se aplican restricciones CHECK o validaciones en el nivel de Service Layer para asegurar que **creditos** nunca sea negativo. Si una transacción intentara dejar el saldo en -100, la lógica de JPA arrojará una **InsufficientFundsException** antes de realizar el *commit*.
- **Isolation (Aislamiento):** Se utiliza el nivel de aislamiento **READ_COMMITTED** (por defecto en MySQL). Esto impide que, si dos procesos intentan comprar el mismo objeto raro simultáneamente, el sistema pueda entrar en un estado de *race condition* (condición de carrera), garantizando que el saldo se actualice secuencialmente y de forma segura.
- **Durability (Durabilidad):** Una vez que el método marcado con @Transactional termina con éxito, los cambios se confirman en el disco del servidor MySQL, garantizando que el progreso es permanente y resistente a reinicios del servidor.

3. Integridad Referencial y Tablas Puente

- **Inventario / Tablas Puente:** Para resolver la relación *Many-to-Many* entre **usuarios** e **items**, se utiliza una tabla puente que almacena la relación (**usuario_id**, **item_id**).
- **Constraints:**
 - **ForeignKey** con restricción **ON DELETE CASCADE**: Si un usuario elimina su cuenta, el sistema limpia automáticamente su inventario y su historial de partidas para evitar datos huérfanos (*Dangling references*).

- **NOT NULL:** Aplicado a campos críticos como `id`, `usuario_id` e `item_id` para garantizar que no existan registros en el inventario que no pertenezcan a un usuario o que no hagan referencia a un objeto real.